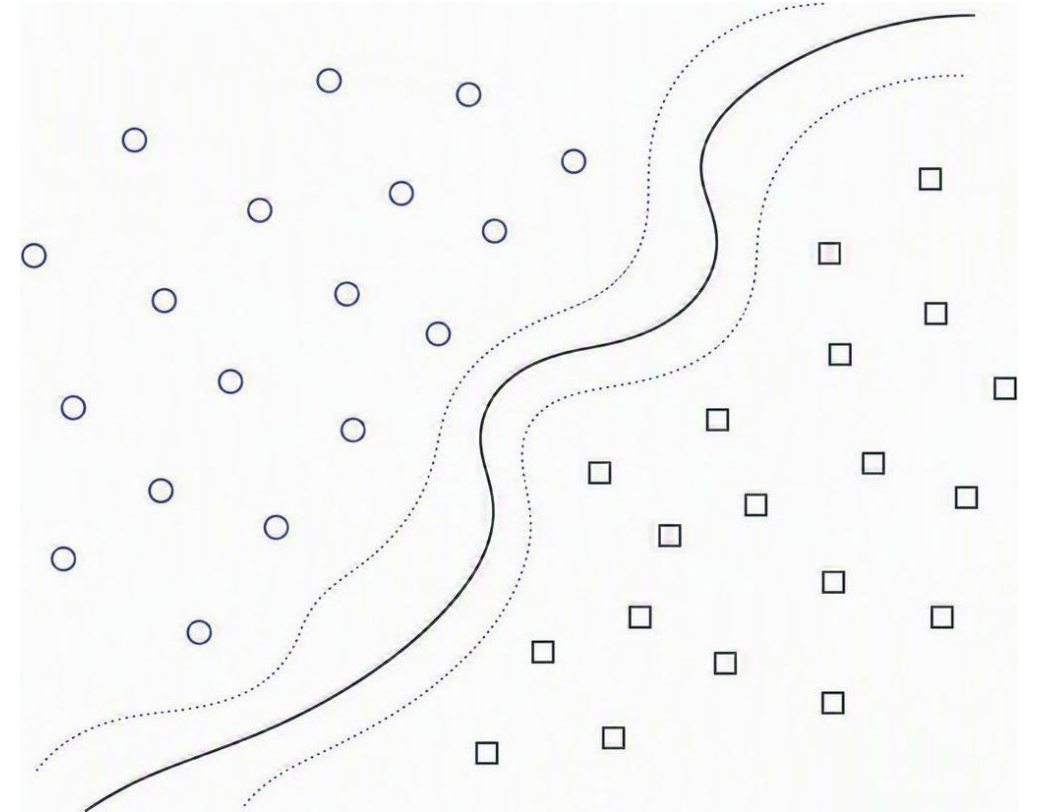


Métricas de clasificación en Machine Learning

Cómo evaluar modelos más allá del accuracy

Ejemplo central: diagnóstico médico binario (enfermo / sano) y clasificación multiclase Iris.



¿Por qué necesitamos métricas?

Entrenar un modelo no alcanza: hay que medir qué tan bien predice — y con la métrica correcta.

- Un modelo puede parecer bueno y fallar justo en los casos importantes.
- Cada problema exige una métrica distinta.
- No todos los errores cuestan lo mismo.
- A veces importa detectar positivos; a veces, evitar falsas alarmas.

Dónde importa la elección de métrica

- Diagnóstico médico
- Fraude financiero
- Detección de spam
- Churn de clientes
- Clasificación de imágenes
- Riesgo crediticio

Predicción correcta vs. incorrecta

Todo se reduce a comparar el valor real con el valor predicho por el modelo.

CORRECTO

Paciente real

Enfermo

El modelo predice

Enfermo

ERROR

Paciente real

Sano

El modelo predice

Enfermo

Clasificación binaria y multiclase

BINARIA

Dos clases

Positivo / **Negativo**

Ejemplo: predecir si un paciente está enfermo o sano.

Muchas métricas se entienden más fácil aquí.

MULTICLASE

Más de dos clases

Setosa **Versicolor** **Virginica**

Ejemplo: clasificar una flor Iris en una de tres especies.

Las métricas se extienden calculándolas por clase.

Matriz de Confusion

		P R E D I C H O	
		Positivo	Negativo
R E A L	Positivo	TP Verdadero Positivo	FN Falso Negativo
	Negativo	FP Falso Positivo	TN Verdadero Negativo

Matriz de confusión: la idea

Muestra, de un vistazo, cómo se reparten aciertos y errores del modelo entre clases.

- Filas = clase real; columnas = clase predicha.
- La diagonal son los aciertos.
- Fuera de la diagonal, los errores.
- Es la base de casi todas las métricas.

REAL

		PREDICHO	
		Positivo	Negativo
REAL	Positivo	TP True Positive	FN False Negative
	Negativo	FP False Positive	TN True Negative

Los cuatro resultados posibles

TP

True Positive

Enfermo detectado como enfermo.

TN

True Negative

Sano detectado como sano.

FP

False Positive

Sano marcado como enfermo (falsa alarma).

FN

False Negative

Enfermo marcado como sano (se escapa).

Ejemplo numérico: 100 pacientes

100 pacientes · 50 enfermos · 50 sanos

		PREDICHO	
		Positivo	Negativo
REAL	Positivo	TP 40 True Positive	FN 10 False Negative
	Negativo	FP 5 False Positive	TN 45 True Negative

Lectura rápida

- 40 enfermos detectados
- 10 enfermos que se escapan
- 5 falsas alarmas
- 45 sanos confirmados

Interpretar los errores, no solo contarlos

- El modelo acierta bastante: 85 de 100 casos.
- Detecta 40 de 50 enfermos.
- Se le escapan 10 enfermos (falsos negativos).
- Genera solo 5 falsas alarmas (falsos positivos).

La gravedad depende del contexto

En medicina, un falso negativo (dejar ir a un enfermo) suele ser mucho más grave que un falso positivo (un chequeo de más).

→ El mismo modelo puede ser bueno o inaceptable según qué error pese más.

En código (sklearn)

Construir la matriz de confusión con datos chicos verificables a mano.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import confusion_matrix

y_true = [1]*10 + [0]*10 # 1=enfermo, 0=sano
y_pred = [1]*8 + [0]*2 + [0]*9 + [1]*1

print(confusion_matrix(y_true, y_pred))
```

SALIDA ESPERADA

```
[[9 1]
 [2 8]]
```

(TN=9, FP=1, FN=2, TP=8)

Accuracy

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

Accuracy

Mide el porcentaje total de predicciones correctas sobre el total de casos.
Es la métrica más intuitiva y la primera que se mira.

La pregunta que responde

¿Qué proporción del total predijo bien?

FÓRMULA

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Predicciones correctas ÷ total de predicciones

Accuracy en el caso base

Con la matriz de los 100 pacientes

$$\text{Accuracy} = (40 + 45) / 100$$

$$= 0.85$$

El modelo acierta el 85% de los casos.

De dónde salen los números

	Valor
TP (aciertos +)	40
TN (aciertos -)	45
FP + FN (errores)	15
Total	100

¿Cuándo sí conviene usar accuracy?

- Cuando las clases están balanceadas.
- Cuando los errores tienen un impacto similar.
- Cuando se busca una métrica general y rápida.
- Como primera visión del rendimiento.

Ejemplos donde funciona

- Clasificación de flores Iris con clases equilibradas.
- Imágenes con distribución de clases similar.
- Problemas iniciales de aprendizaje y prototipos.

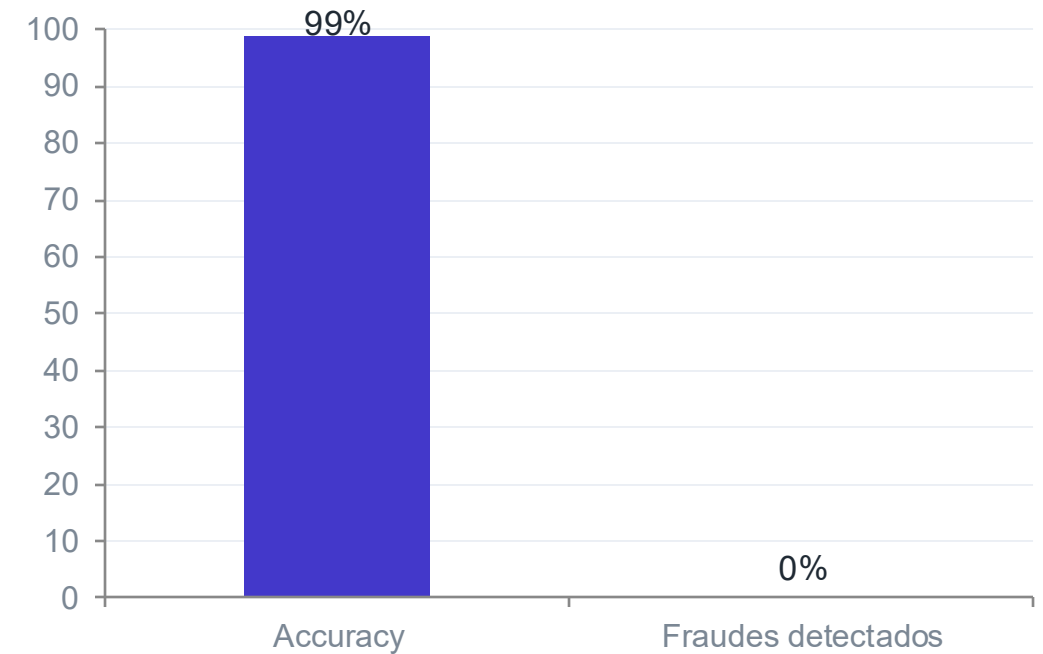
Cuándo accuracy engaña: el desbalanceo

1000 transacciones · 990 normales · 10 fraudes

Un modelo que siempre predice “normal”

99%

de accuracy... y 0 fraudes detectados.



Un accuracy alto no siempre significa un buen modelo.

Precision

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$



Precision

De todo lo que el modelo marcó como positivo, mide qué fracción realmente lo era. Castiga los falsos positivos.

La pregunta que responde

De lo que predije como positivo, ¿cuánto era realmente positivo?

FÓRMULA

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Mide la calidad de las alarmas del modelo

Precision en el caso base

De los que el modelo llamó “enfermos”

$$\text{Precision} = 40 / (40 + 5)$$

$$= 0.89$$

Cuando dice “positivo”, acierta ~89% de las veces.

Solo entran en juego dos celdas

$$\text{TP} = 40$$

aciertos positivos

$$\text{FP} = 5$$

falsas alarmas

¿Cuándo priorizar precision?

Cuando una falsa alarma tiene un costo alto. Queremos evitar falsos positivos.

1

Marcar una transacción como fraude

2

Bloquear una cuenta de usuario

3

Tratamiento agresivo tras un diagnóstico

4

Enviar una alerta crítica

5

Clasificar un email legítimo como spam

Precision es clave cuando una falsa alarma cuesta caro: dinero, confianza o un tratamiento innecesario.

Recall
Sensibility
TPR (True Possitive Rates)

RECALL

Recall

De todos los positivos reales, mide cuántos logró encontrar el modelo. También se llama sensibilidad o true positive rate. Castiga los falsos negativos.

La pregunta que responde

De todos los positivos reales, ¿cuántos logré detectar?

FÓRMULA

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Sensibilidad · True Positive Rate

Recall en el caso base

De todos los enfermos reales

$$\text{Recall} = 40 / (40 + 10)$$

$$= 0.80$$

Detecta el 80% de los pacientes enfermos.

Precision y recall miran cosas distintas

$$\text{TP} = 40$$

enfermos detectados

$$\text{FN} = 10$$

enfermos que se escapan

¿Cuándo priorizar recall?

Cuando dejar pasar un positivo real es peligroso o costoso. Queremos evitar falsos negativos.

1

Diagnóstico de una enfermedad grave

2

Detección de fraude

3

Detección de fallas críticas

4

Anticipar el abandono de clientes

5

Riesgos de seguridad

Recall es clave cuando el positivo no detectado es lo peligroso: un enfermo sin diagnosticar, un fraude que pasa.

Specificity

Especificidad

TNR (True Negative Rate)

$$\text{Specificity} = \frac{\text{True negative}}{\text{True negative} + \text{false positive}}$$

Specificity

De todos los negativos reales, mide cuántos identificó bien el modelo. También se llama especificidad o true negative rate. Castiga los falsos positivos.

La pregunta que responde

De todos los negativos reales, ¿cuántos descarté bien?

FÓRMULA

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Especificidad · True Negative Rate

Specificity en el caso base

De todos los sanos reales

$$\text{Specificity} = 45 / (45 + 5)$$

$$= 0.90$$

Descarta bien al 90% de los pacientes sanos.

Specificity mira la clase negativa

$$\text{TN} = 45$$

sanos descartados bien

$$\text{FP} = 5$$

sanos marcados como enfermos

Recall vs. Specificity

Son métricas complementarias: una vigila los positivos, la otra los negativos.

SENSIBILIDAD · RECALL

$$\text{Recall} = TP / (TP + FN)$$

De los positivos reales (enfermos), cuántos detecté.

ESPECIFICIDAD · SPECIFICITY

$$\text{Specificity} = TN / (TN + FP)$$

De los negativos reales (sanos), cuántos descarté bien.

LA CONEXIÓN CON LA CURVA ROC

El eje Y de la ROC es Recall (TPR) y el eje X es $FPR = 1 - \text{Specificity}$. Por eso, al mover el umbral, subir una suele bajar la otra.

En código (sklearn)

La specificity no viene directa: se calcula desde la matriz.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import confusion_matrix

y_true = [1]*10 + [0]*10
y_pred = [1]*8 + [0]*2 + [0]*9 + [1]*1

tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
specificity = tn / (tn + fp)
print("Specificity:", specificity)
```

SALIDA ESPERADA

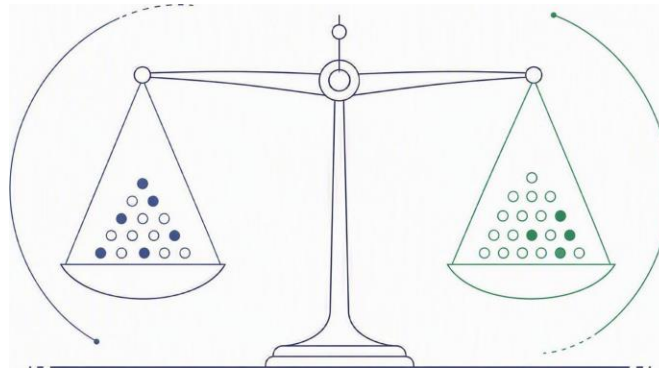
Specificity: 0.9

Precision vs. Recall

PRECISION

De lo que predije como positivo, cuánto era realmente positivo.

Castiga los **falsos positivos**



RECALL

De lo que era positivo, cuánto logré encontrar.

Castiga los **falsos negativos**

Ejemplo intuitivo: detectar fraude

MODELO A · conservador

Marca pocos casos como fraude.

- **Precision alta**
- **Recall bajo: se le escapan fraudes**

“Solo alarmo cuando estoy muy seguro.”

MODELO B · agresivo

Marca muchos casos como fraude.

- **Recall alto: casi no se escapa nada**
- **Precision baja: muchas falsas alarmas**

“Prefiero revisar de más antes que perder un fraude.”



F1 Score

F1 Score

Combina precision y recall en una sola métrica mediante su media armónica. Solo es alto cuando ambas lo son.

La pregunta que responde

¿Hay un buen equilibrio entre precision y recall?

FÓRMULA

$$F1 = 2 \times \frac{P \times R}{P + R}$$

Media armónica de Precision y Recall

F1 Score en el caso base

Combinando precision y recall

$$F1 = 2 \times (0.89 \times 0.80) / (0.89 + 0.80)$$

≈ 0.84

Buen equilibrio entre detectar y no alarmar de más.

Las tres métricas del caso base

Precision **0.89**

Recall **0.80**

F1 Score **0.84**

¿Cuándo usar F1 Score?

- Cuando las clases están desbalanceadas.
- Cuando importan tanto los falsos positivos como los negativos.
- Cuando se necesita una métrica resumida y única.
- Cuando accuracy puede ser engañoso.

Casos típicos

- Fraude
- Churn de clientes
- Diagnóstico médico
- Clasificación de reclamos
- Detección de anomalías etiquetadas

En código (sklearn)

Las cuatro métricas base a partir de las mismas predicciones.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_true = [1]*10 + [0]*10
y_pred = [1]*8 + [0]*2 + [0]*9 + [1]*1

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
```

SALIDA ESPERADA

```
Accuracy: 0.85
Precision: 0.8888888888888888
Recall: 0.8
F1: 0.8421052631578948
```

En código (sklearn)

Todo junto en un solo reporte por clase.

CÓDIGO (SKLEARN)

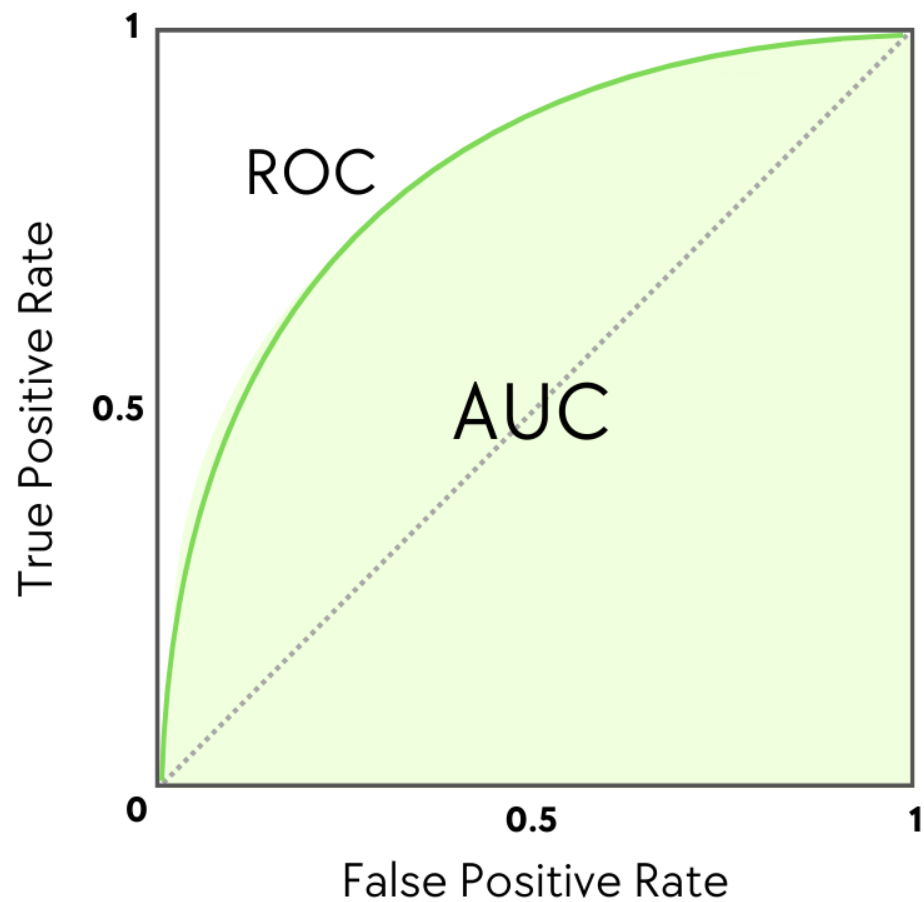
```
from sklearn.metrics import classification_report

y_true = [1]*10 + [0]*10
y_pred = [1]*8 + [0]*2 + [0]*9 + [1]*1

print(classification_report(y_true, y_pred, target_names=["Sano", "Enfermo"]))
```

SALIDA ESPERADA

	precision	recall	f1-score	support
Sano	0.82	0.90	0.86	10
Enfermo	0.89	0.80	0.84	10
accuracy			0.85	20
macro avg	0.85	0.85	0.85	20
weighted avg	0.85	0.85	0.85	20



ROC Curve

ROC Curve: rendimiento según el umbral

Muestra cómo cambia el rendimiento del modelo al mover el umbral de decisión, recorriendo todos los umbrales posibles.

Eje Y

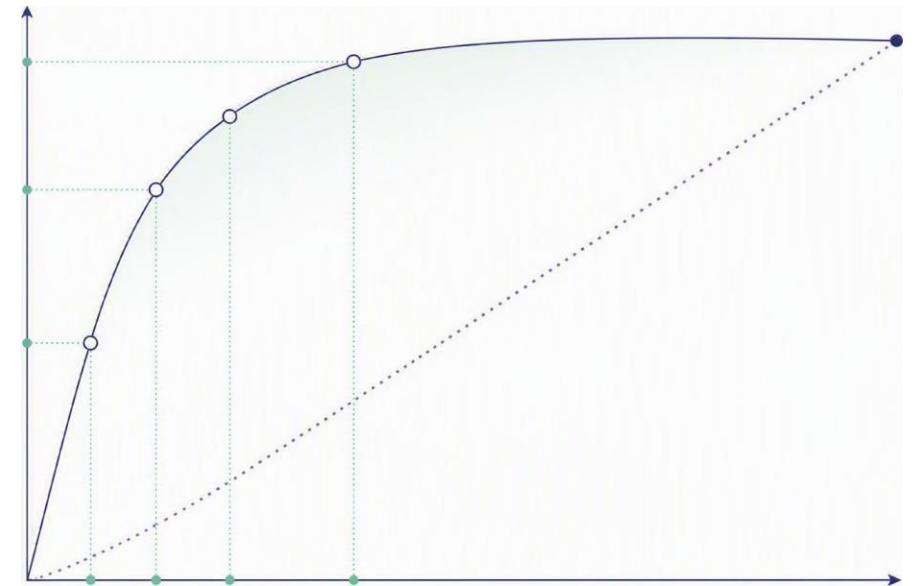
True Positive Rate

= Recall: positivos bien detectados

Eje X

False Positive Rate

Sanos marcados como positivos por error



El umbral de decisión

Muchos modelos devuelven una probabilidad, no una etiqueta. El umbral decide dónde cortar.

Probabilidad de enfermedad estimada por el modelo

= **0.70**



Umbral = 0.50

$0.70 \geq 0.50 \rightarrow$ clasifica como POSITIVO

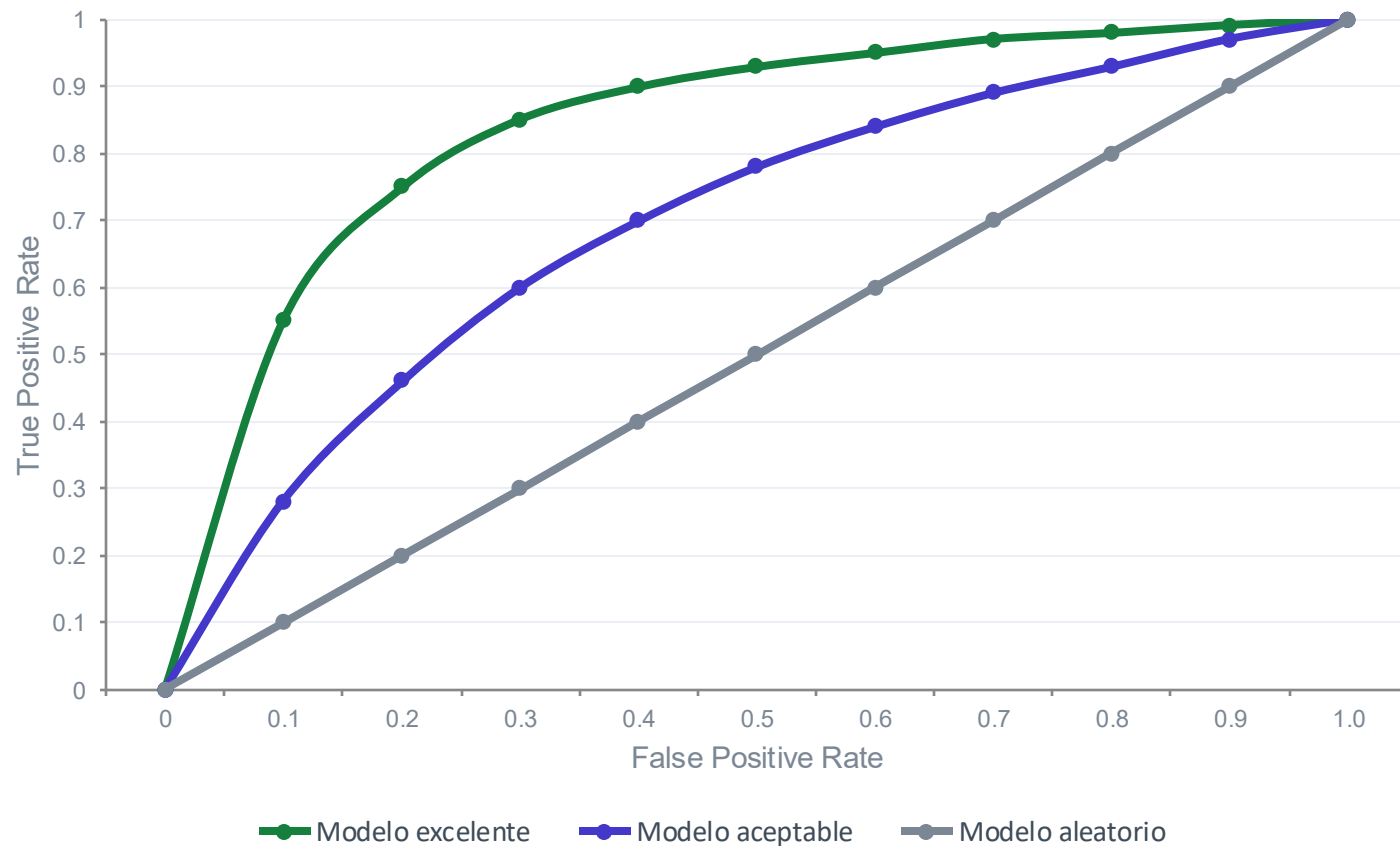
Umbral bajo $\rightarrow$ más recall, más falsos positivos.

Umbral = 0.80

$0.70 < 0.80 \rightarrow$ clasifica como NEGATIVO

Umbral alto $\rightarrow$ más precision, más falsos negativos.

Leer una curva ROC



Cómo interpretarla

Aleatorio: diagonal, no distingue clases.

Aceptable: se despega hacia arriba.

Excelente: abraza la esquina superior izquierda.

Cuanto más arriba y a la izquierda, mejor separa el modelo.

En código (sklearn)

Mover el umbral cambia el balance entre precision y recall.

CÓDIGO (SKLEARN)

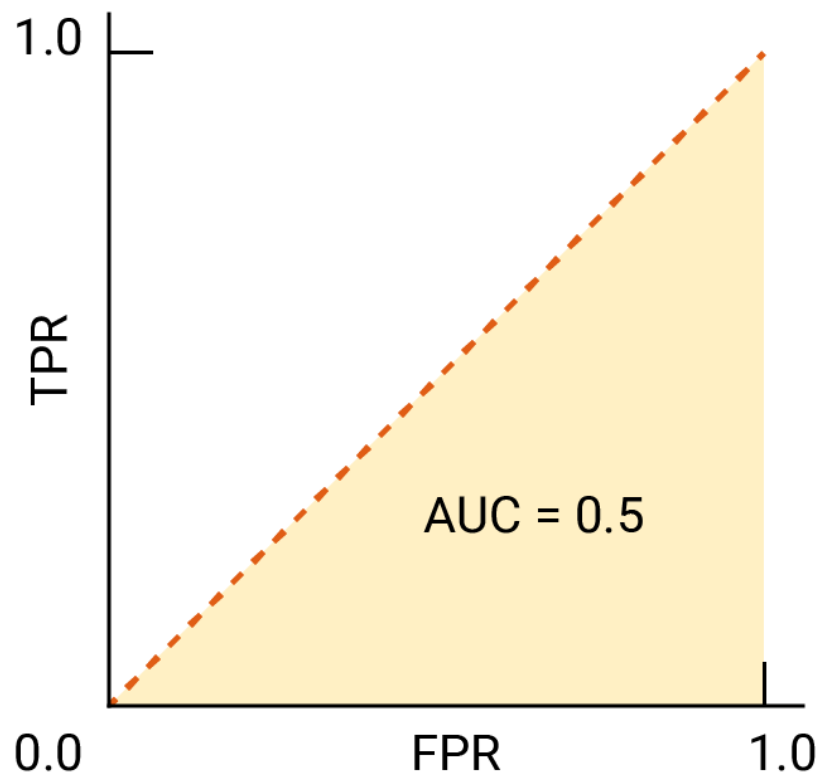
```
from sklearn.metrics import precision_score, recall_score

y_true = [1,1,1,1,0,0,0,0]
y_proba = [0.9,0.8,0.4,0.3,0.6,0.35,0.2,0.1]

for umbral in [0.3, 0.5]:
    y_pred = [1 if p >= umbral else 0 for p in y_proba]
    print(f"Umbral {umbral} -> Precision: {precision_score(y_true, y_pred):.3f}, Recall: {recall_score(y_true, y_pred):.3f}")
```

SALIDA ESPERADA

```
Umbral 0.3 -> Precision: 0.667, Recall: 1.000
Umbral 0.5 -> Precision: 0.667, Recall: 0.500
```



AUC

ROC AUC: la curva en un número

AUC = Area Under the Curve. Resume toda la curva ROC en un único valor entre 0.5 y 1.0.

0.5

Como el azar

0.7

Aceptable

0.8

Bueno

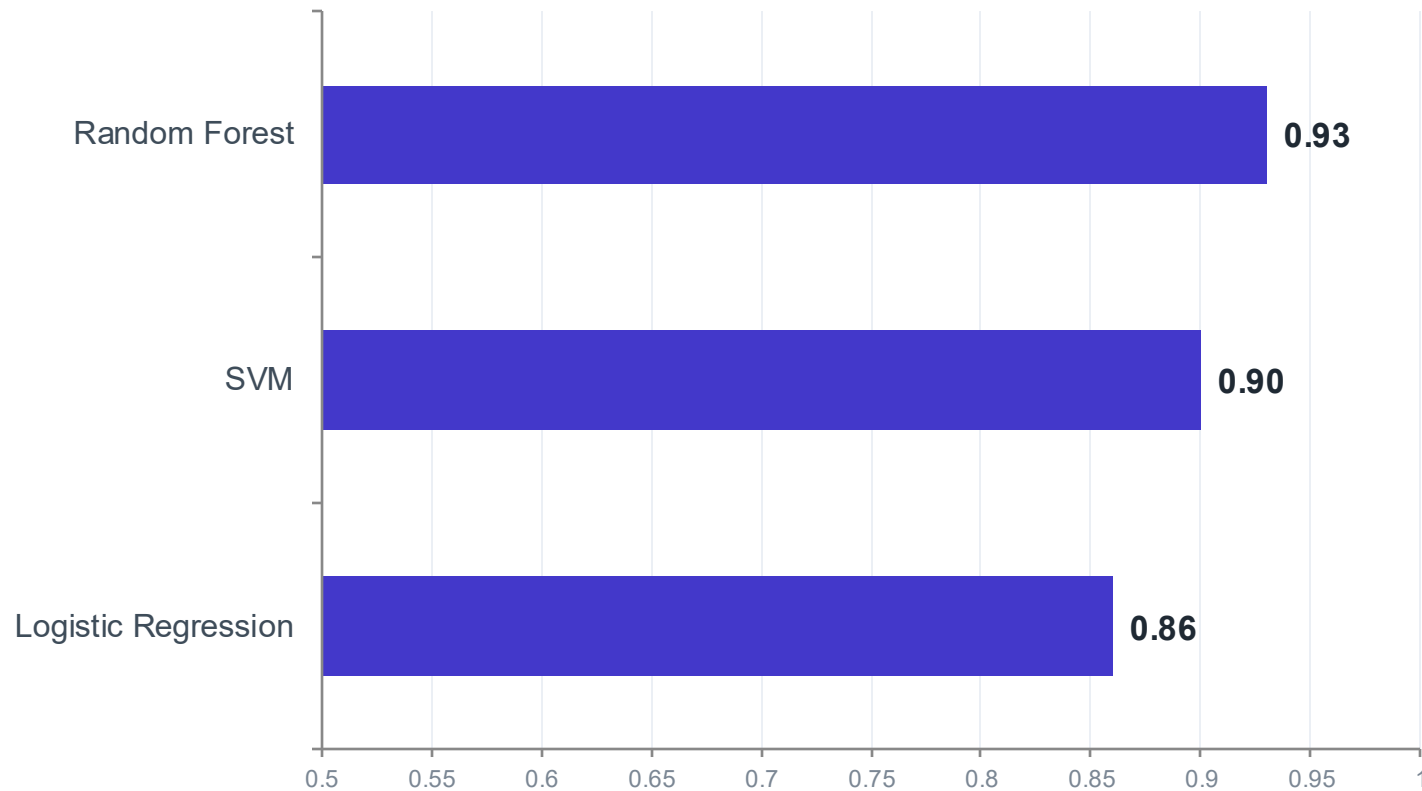
0.9+

Muy bueno

1.0

Separación perfecta

Comparar modelos por ROC AUC



Interpretación

Random Forest (0.93) separa mejor las clases que SVM (0.90) y Logistic Regression (0.86) en este ejemplo.

Valores ilustrativos: dependen del dataset.

¿Cuándo usar ROC AUC?

- Cuando el modelo entrega probabilidades o scores.
- Cuando se quiere evaluar la capacidad de separación.
- Cuando se comparan modelos sin fijar un umbral.
- Sobre todo en clasificación binaria.

Cuidado con el desbalanceo

Con clases muy desbalanceadas, el ROC AUC puede verse optimista.

En esos casos, mirar también Precision, Recall, F1 y la Precision-Recall Curve.

En código (sklearn)

El AUC resume la curva ROC en un solo número.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import roc_auc_score

y_true = [1,1,1,1,0,0,0,0]
y_proba = [0.9,0.8,0.4,0.3,0.6,0.35,0.2,0.1]

print("AUC:", roc_auc_score(y_true, y_proba))
```

SALIDA ESPERADA

AUC: 0.8125

Precision-Recall Curve: el foco en los positivos

Recorre todos los umbrales, como la curva ROC, pero enfrenta Precision contra Recall. Es más honesta que el ROC AUC cuando la clase positiva es rara.

Eje Y

Precision

De lo que predigo positivo, qué porción realmente lo es.

Eje X

Recall

De los positivos reales, cuántos logro detectar.



En código (sklearn)

El Average Precision resume la curva Precision-Recall.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import average_precision_score

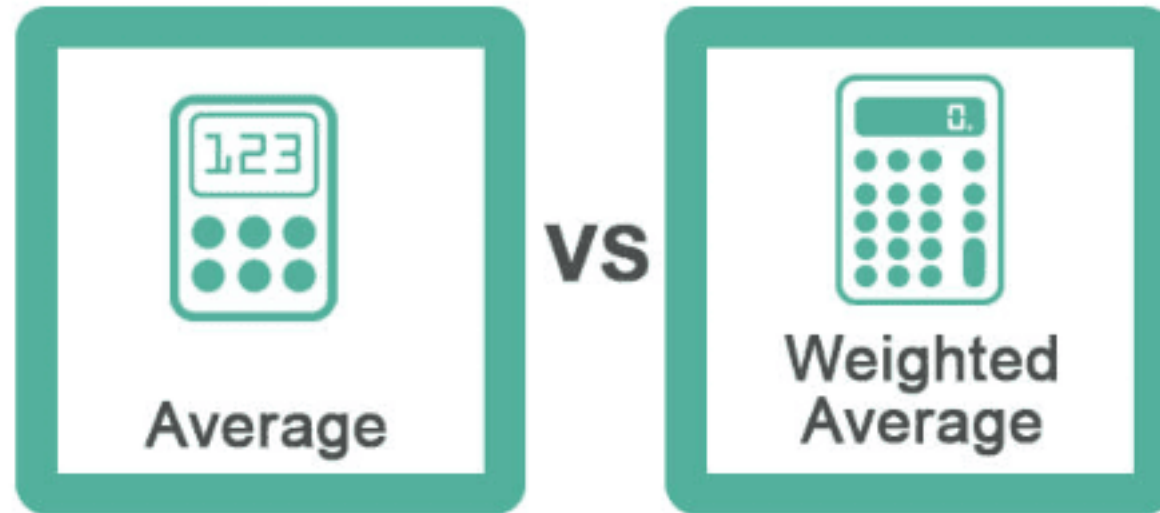
y_true = [1,1,1,1,0,0,0,0]
y_proba = [0.9,0.8,0.4,0.3,0.6,0.35,0.2,0.1]

print("Average Precision:", average_precision_score(y_true, y_proba))
```

SALIDA ESPERADA

Average Precision: 0.854

Macro Average & Weighted Average



Matriz de confusión multiclase (Iris)

En multiclase la matriz se vuelve N×N: una fila y una columna por clase. La diagonal son los aciertos; fuera de ella, las confusiones.

La diagonal

Aciertos

Cada clase predicha bien. Setosa: 50 de 50 correctas.

Fuera de ella

Confusiones

Versicolor y Virginica se confunden entre sí.

		PREDICHO		
		Setosa	Versicolor	Virginica
REAL	Setosa	50	0	0
	Versicolor	0	39	11
	Virginica	0	17	33

Valores ilustrativos · 150 flores (50 por clase)

Métricas en clasificación multiclase

En multiclase, precision, recall y F1 se calculan por clase (una-vs-resto). Luego hay que resumirlas.

Clase (Iris)	Precision	Recall	F1
Setosa	1.00	1.00	1.00
Versicolor	0.82	0.78	0.80
Virginica	0.74	0.67	0.70

¿Y ahora?

Tenemos una F1 por clase. Para un único número comparamos dos formas de promediar:

Macro · Weighted

Macro Average: todas las clases pesan igual

Promedio simple de la métrica entre clases, sin mirar cuántos casos tiene cada una.

Setosa **F1 = 1.00**

Versicolor **F1 = 0.80**

Virginica **F1 = 0.70**

Macro F1 = (1.00 + 0.80 + 0.70) / 3

= 0.83

Qué significa

Todas las clases pesan lo mismo, aunque tengan muy distinta cantidad de casos.

Una clase minoritaria con bajo F1 arrastra el promedio hacia abajo.

¿Cuándo usar Macro Average?

- Cuando todas las clases son igual de importantes.
- Cuando queremos detectar si el modelo falla en una clase minoritaria.
- Cuando no queremos que las clases grandes dominen el promedio.

Ejemplo

Clasificación de enfermedades raras y comunes, donde acertar en la enfermedad rara importa tanto como en la común.

Weighted Average: pondera por frecuencia

Promedio ponderado por la cantidad de ejemplos de cada clase: las clases grandes influyen más.

Clase	Casos	F1	Peso
A	80	0.95	0.80
B	15	0.70	0.15
C	5	0.40	0.05

$$\text{Weighted F1} = 0.80 \cdot 0.95 + 0.15 \cdot 0.70 + 0.05 \cdot 0.40 \approx 0.89$$

La clase A (80 casos) domina el resultado.

Macro vs. Weighted

Macro F1 = 0.68

todas las clases igual

Weighted F1 = 0.89

domina la clase grande

¿Cuándo usar Weighted Average?

- Cuando queremos una métrica global fiel a la distribución real del dataset.
- Cuando las clases tienen tamaños muy distintos.
- Cuando importa el rendimiento promedio según la frecuencia de cada clase.

Cuidado

El weighted average puede ocultar un mal rendimiento en las clases minoritarias, porque las grandes dominan el promedio.

En código (sklearn)

Macro promedia por clase; weighted pondera por soporte.

CÓDIGO (SKLEARN)

```
from sklearn.metrics import classification_report, f1_score

y_true = ['A']*10 + ['B']*3 + ['C']*2
y_pred = ['A']*10 + ['A','B','B'] + ['B','C']

print(classification_report(y_true, y_pred))
print("Macro F1:", f1_score(y_true, y_pred, average='macro'))
print("Weighted F1:", f1_score(y_true, y_pred, average='weighted'))
```

SALIDA ESPERADA

	precision	recall	f1-score	support
A	0.91	1.00	0.95	10
B	0.67	0.67	0.67	3
C	1.00	0.50	0.67	2
accuracy			0.87	15
macro avg	0.86	0.72	0.76	15
weighted avg	0.87	0.87	0.86	15

Macro F1: 0.762
Weighted F1: 0.857

Tiempo de Entrenamiento y Costo Computacional



Más allá de la métrica: el costo

La calidad predictiva no es lo único que importa para elegir un modelo.

01

Cuánto tarda en entrenar

02

Cuánto tarda en predecir

03

Cuánto cuesta computacionalmente

04

Si escala bien con más datos

05

Si es viable en producción

06

**Si permite reentrenamientos
frecuentes**

Tiempo de entrenamiento vs. calidad

MODELO A

Accuracy 0.91

Entrenamiento: **1 segundo**

MODELO B

Accuracy 0.92

Entrenamiento: **3 minutos**

¿Vale la pena ganar 1 punto porcentual si el costo computacional se multiplica por 180?

Rendimiento y costo, lado a lado

Modelo	Accuracy	F1	ROC AUC	Tiempo de entrenamiento
Logistic Regression	0.94	0.93	0.96	Muy bajo
KNN	0.93	0.92	0.95	Bajo al entrenar, alto al predecir
Decision Tree	0.91	0.90	0.91	Bajo
Random Forest	0.96	0.95	0.98	Medio
SVM RBF	0.95	0.94	0.97	Medio / alto
Gradient Boosting	0.97	0.96	0.98	Alto

Valores ilustrativos: el rendimiento y los tiempos dependen del dataset, el hardware y el ajuste.

Modelos rápidos vs. costosos

Generalmente rápidos

- Logistic Regression
- Naive Bayes
- Decision Tree

Intermedios

- Random Forest
- KNN
- SVM lineal

Más costosos

- SVM RBF (datos grandes)
- Gradient Boosting
- XGBoost / LightGBM
- Redes neuronales

LightGBM suele ser eficiente en datasets grandes, pero sigue siendo más complejo que un modelo lineal simple.

¿Cuál modelo parece ser el mejor?



Comparación de Modelos

Comparar en varias dimensiones

Un buen modelo se elige mirando muchos ejes a la vez, no una sola métrica.

Accuracy

Precision

Recall

F1 Score

ROC AUC

Matriz de confusión

Tiempo de entrenamiento

Tiempo de predicción

Interpretabilidad

Riesgo de overfitting

Facilidad de ajuste

Escalabilidad

Tabla comparativa de modelos

Modelo	Acc.	Prec.	Rec.	F1	AUC	Tiempo	Interpret.	Comentario práctico
Logistic Regression	0.94	0.93	0.93	0.93	0.96	Muy bajo	Alta	Rápido y explicable; gran línea base.
KNN	0.93	0.92	0.93	0.92	0.95	Bajo*	Media	Predicción lenta con muchos datos.
Decision Tree	0.91	0.90	0.90	0.90	0.91	Bajo	Alta	Interpretable; propenso a overfitting.
Random Forest	0.96	0.95	0.95	0.95	0.98	Medio	Media	Robusto y preciso; menos explicable.
SVM	0.95	0.94	0.93	0.94	0.97	Medio/alto	Baja	Fuerte en datasets pequeños.
Naive Bayes	0.88	0.87	0.89	0.88	0.90	Muy bajo	Media	Muy rápido; asume independencia.
Gradient Boosting	0.97	0.96	0.96	0.96	0.98	Alto	Baja	Top performance; ajuste costoso.
MLP	0.95	0.94	0.95	0.94	0.97	Alto	Baja	Flexible; necesita datos y tuning.

* KNN entrena rápido pero predice lento. Valores ficticios pero realistas, con fines didácticos.

Interpretar la comparación

Random Forest supera a Logistic Regression en F1 y ROC AUC... pero tarda más y es menos interpretable. ¿Cuál elegir? Depende del objetivo.

SI ...

Necesito explicar

Logistic Regression: transparente y auditable.

SI ...

Priorizo performance

Random Forest o Gradient Boosting.

SI ...

Dataset pequeño

SVM, Logistic Regression o Naive Bayes.

Guía rápida: ¿qué métrica elegir?

Clases balanceadas	➤	Accuracy
Clases desbalanceadas	➤	F1 · Precision · Recall
No perder positivos	➤	Recall
Evitar falsas alarmas	➤	Precision
Evaluar separación general	➤	ROC AUC
Multiclase	➤	Macro y Weighted average
Pensando en producción	➤	+ Tiempo de entren. y predicción

Casos prácticos de selección

CASO 1

Enfermedad grave

Métricas clave

Recall

CASO 2

Fraude financiero

Métricas clave

Recall · Precision · F1

CASO 3

Filtro de spam

Métricas clave

Precision · F1

CASO 4

Flores Iris

Métricas clave

Accuracy · F1 macro · matriz

CASO 5

Producción diaria

Métricas clave

F1 · ROC AUC · tiempo

Errores comunes al evaluar

✗ Mirar solo el accuracy

✗ Olvidar el tiempo de entrenamiento

✗ No revisar la matriz de confusión

✗ Elegir el modelo más complejo sin necesidad

✗ Ignorar las clases desbalanceadas

✗ No interpretar los errores según el negocio

✗ No comparar precision y recall

Mensajes clave

01

Accuracy no siempre alcanza

engaña con clases desbalanceadas

02

La matriz de confusión es la base

revela el tipo de error, no solo cuántos

03

Precision y recall

responden preguntas distintas

04

F1 Score

equilibra precision y recall en un número

05

ROC AUC

mide la capacidad de separación

06

Macro vs. Weighted

trato igualitario vs. peso por frecuencia

EVALUAR ES ENTENDER LOS ERRORES

Evaluar un modelo es entender sus errores

“Un buen modelo de clasificación no es el que más acierta, sino el que comete los errores menos costosos para el problema que queremos resolver.”

